

Déploiement du modèle de prédiction du turnover

Mission Futurisys — Projet 5

Elodie Chen

Juillet 2026

Contexte et objectif

Problématique



Fururisys souhaite rendre ses modèles de machine learning opérationnels et accessibles via une API performante



Un modèle LightGBM a été entraîné sur des données RH (SIRH, évaluations, sondage de satisfaction).

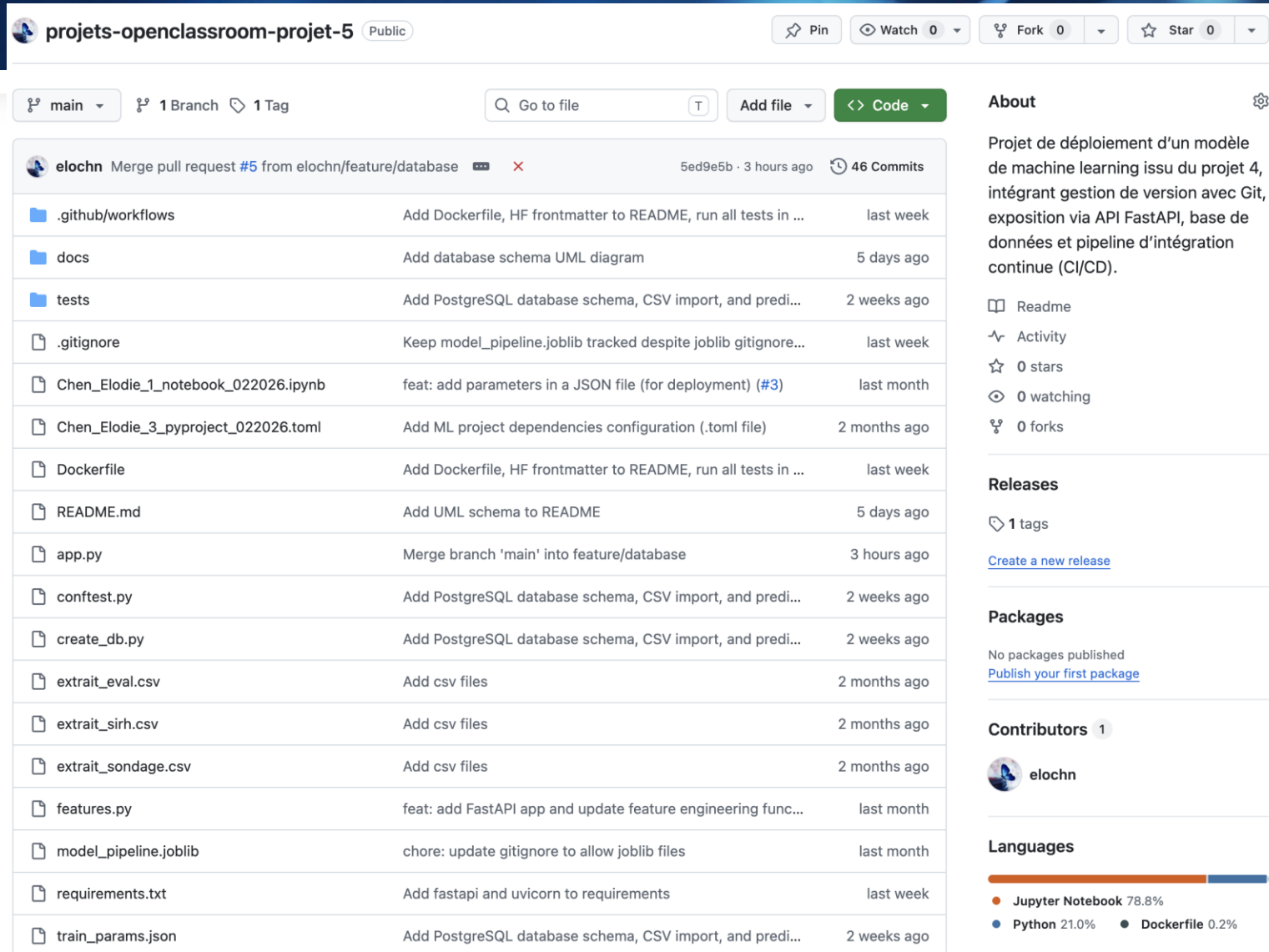


Le modèle que l'on veut exposer permet d'anticiper les départs volontaires des employés pour réduire les coûts de recrutement et de formation.

Objectif du projet

- Mettre ce modèle en production :
- Exposer le modèle via une **API REST**
- Journaliser chaque prédiction en **base de données**
- Automatiser les tests et le **déploiement CI/CD**
- Rendre l'API accessible en ligne via **Hugging Face**

Structure du dépôt Git



projets-openclassroom-projet-5 Public

main 1 Branch 1 Tag

Go to file T Add file <> Code

elochn Merge pull request #5 from elochn/feature/database 5ed9e5b · 3 hours ago 46 Commits

.github/workflows	Add Dockerfile, HF frontmatter to README, run all tests in ...	last week
docs	Add database schema UML diagram	5 days ago
tests	Add PostgreSQL database schema, CSV import, and predi...	2 weeks ago
.gitignore	Keep model_pipeline.joblib tracked despite joblib gitignore...	last week
Chen_Elodie_1_notebook_022026.ipynb	feat: add parameters in a JSON file (for deployment) (#3)	last month
Chen_Elodie_3_pyproject_022026.toml	Add ML project dependencies configuration (.toml file)	2 months ago
Dockerfile	Add Dockerfile, HF frontmatter to README, run all tests in ...	last week
README.md	Add UML schema to README	5 days ago
app.py	Merge branch 'main' into feature/database	3 hours ago
confest.py	Add PostgreSQL database schema, CSV import, and predi...	2 weeks ago
create_db.py	Add PostgreSQL database schema, CSV import, and predi...	2 weeks ago
extrait_eval.csv	Add csv files	2 months ago
extrait_sirh.csv	Add csv files	2 months ago
extrait_sondage.csv	Add csv files	2 months ago
features.py	feat: add FastAPI app and update feature engineering func...	last month
model_pipeline.joblib	chore: update gitignore to allow joblib files	last month
requirements.txt	Add fastapi and uvicorn to requirements	last week
train_params.json	Add PostgreSQL database schema, CSV import, and predi...	2 weeks ago

About

Projet de déploiement d'un modèle de machine learning issu du projet 4, intégrant gestion de version avec Git, exposition via API FastAPI, base de données et pipeline d'intégration continue (CI/CD).

Readme Activity 0 stars 0 watching 0 forks

Releases

1 tags [Create a new release](#)

Packages

No packages published [Publish your first package](#)

Contributors 1

 elochn

Languages

Jupyter Notebook 78.8% Python 21.0% Dockerfile 0.2%

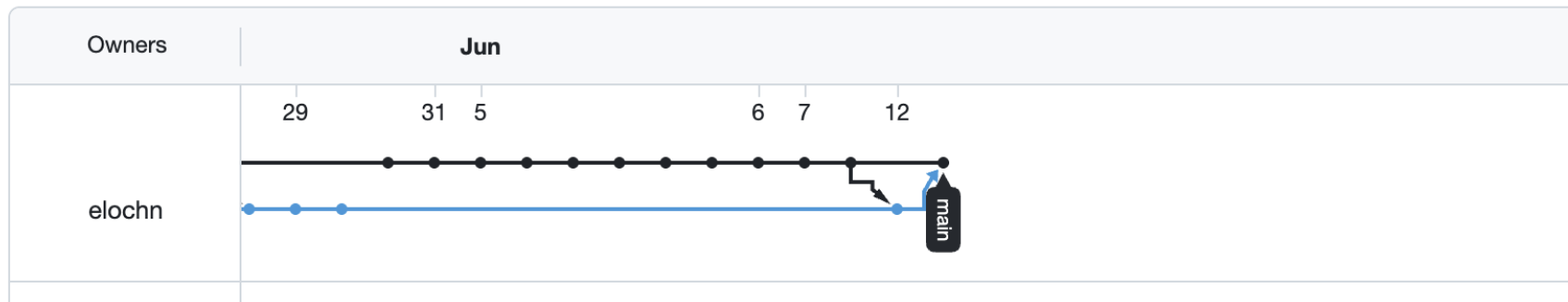
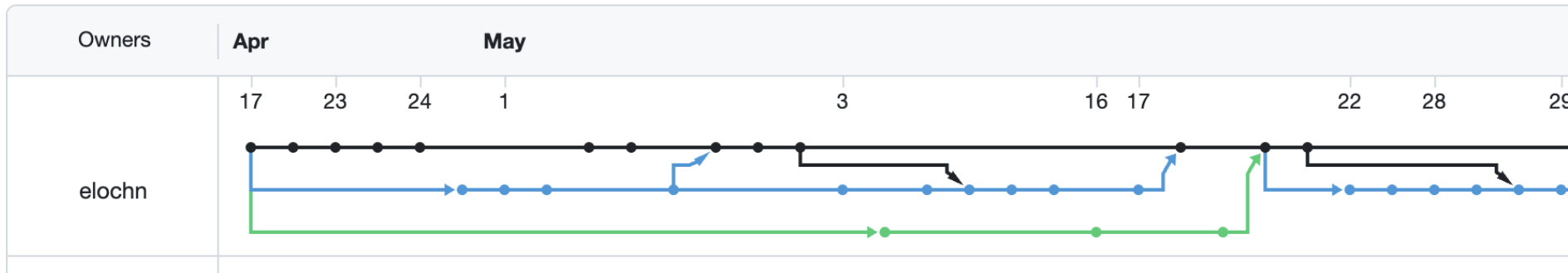
```
proj5/  
├── docs/  
│   └── schema_bdd.png  
├── tests/  
│   ├── test_db.py  
│   └── test_model.py  
├── app.py  
├── create_db.py  
├── features.py  
├── Dockerfile  
├── requirements.txt  
├── train_params.json  
├── model_pipeline.joblib  
├── confest.py  
└── README.md
```

```
└── .github/  
    └── workflows  
        └── ci.yml
```


Historique des branches — GitHub Network

Network graph

Timeline of the most recent commits to this repository and its network ordered by most recently pushed to and updated daily.



Branches créées

- main — production
- feature/api
- feature/database
- feature/tests
- feature/notebook
- feature/readme

Pipeline CI/CD — GitHub Actions



Hugging Face

.github > workflows > ci.yml

```
1 name: CI Pipeline
2
3 on:
4   push:
5     branches: ["main", "feature/notebook", "feature/tests"]
6   pull_request:
7     branches: ["main"]
8
9 jobs:
10  test:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout repository
15        uses: actions/checkout@v4
16
17      - name: Set up Python
18        uses: actions/setup-python@v5
19        with:
20          python-version: '3.11'
21
22      - name: Install dependencies
23        run: |
24          python -m pip install --upgrade pip
25          pip install -r requirements.txt
26          pip install pytest
27
28      - name: Verify that the files exist
29        run: |
30          ls -la
31          echo "Pipeline CI OK ✅"
32
33      - name: Run tests
34        run: |
35          pytest tests/
36        env:
37          ENV: test
```

.github > workflows > ci.yml

```
40 deploy:
41   needs: test
42   runs-on: ubuntu-latest
43   if: github.ref == 'refs/heads/main'
44
45   steps:
46     - name: Checkout repository
47       uses: actions/checkout@v4
48
49     - name: Deploy to Hugging Face Spaces
50       env:
51         HF_TOKEN: ${ secrets.HF_TOKEN }
52       run: |
53         git config --global user.email "ci@github.com"
54         git config --global user.name "GitHub Actions"
55
56         git clone https://huggingface.co/spaces/elch99/projet5 hf-space
57
58         rm -rf hf-space/*
59
60         rsync -av \
61           --exclude='.git' \
62           --exclude='hf-space' \
63           ./ hf-space/
64
65         cd hf-space
66         git add .
67         git commit -m "Deploy from GitHub Actions"
68         git push https://user:$HF_TOKEN@huggingface.co/spaces/elch99/projet5
```

① Job : test

- S'exécute à chaque push sur main
- Installe les dépendances
- Lance pytest tests/
- Bloque le déploiement si un test échoue

② Job : deploy

- Ne s'exécute que si les tests passent
- Copie le code vers Hugging Face Spaces
- Utilise HF_TOKEN (secret GitHub chiffré)

Gestion des secrets

- HF_TOKEN — stocké dans les secrets GitHub
- DATABASE_URL — variable d'environnement
- API_KEY — secret dans Hugging Face Space

API FastAPI — Démonstration

1. Sauvegarde du modèle entraîné avec Joblib

```
from joblib import dump
dump(pipeline, "model_pipeline.joblib")
```

2. Recharge du modèle dans le app.py

```
from joblib import load
pipeline = load("model_pipeline.joblib")
```

Endpoints

- GET / — vérification que l'API tourne
- POST /predict — prédiction du risque de churn

Validation des données — Pydantic

- Valide automatiquement les 27 variables d'entrée avant toute prédiction.

Exemple de réponse

```
{
  "prediction": 1,
  "probabilite_churn": 0.847,
  "interpretation": "Risque de churn"
}
```

Lien : <https://elch99-projet5.hf.space/docs>

API_KEY — secret dans Hugging Face Space

En local : `uvicorn app:app --reload`

Lien : <http://127.0.0.1:8000/docs>

The screenshot shows the documentation for the 'Churn Prediction API' version 0.1.0, which is OAS 3.1 compliant. The page is titled 'Churn Prediction API' and includes a link to the openapi.json file. There is an 'Authorize' button in the top right corner. The 'default' section lists two endpoints: a GET endpoint for the root and a POST endpoint for '/predict'. The 'Schemas' section lists three schemas: 'EmployeeData' (object), 'HTTPValidationError' (object), and 'ValidationError' (object). The page is displayed in a web browser with multiple tabs open, including 'Churn Prediction API - Swagger' and 'Déployez un modèle de Machi'.

Base de données PostgreSQL

2 tables créées via SQLAlchemy

- **employees** — 1 470 employés importés depuis les CSV
- **prediction_logs** — chaque appel à /predict est enregistré avec les 27 inputs, la prédiction, la probabilité et l'horodatage

Sécurité

- DATABASE_URL jamais écrite dans le code
- Lue depuis les variables d'environnement
- Valeur par défaut pour le développement local
- Secret chiffré en production

```
(.venv) elodiechen@macbook-air15 Exercise % psql churn_db -c "SELECT id, created_at, prediction, probabilite_churn FROM prediction_logs LIMIT 5;"
```

id	created_at	prediction	probabilite_churn
1	2026-05-28 13:34:11.808886	0	0.4829039938454827

(1 row)

employees	
id	integer
id_employee	integer NN
age	float
genre	varchar
revenu_mensuel	integer
statut_marital	varchar
departement	varchar
poste	varchar
nombre_experiences_precedentes	integer
annee_experience_totale	float
annees_dans_l_entreprise	integer
annees_dans_le_poste_actuel	integer
satisfaction_employee_environnement	integer
note_evaluation_precedente	integer
niveau_hierarchique_poste	integer
satisfaction_employee_nature_travail	integer
satisfaction_employee_equipe	integer
satisfaction_employee_equilibre_pro_perso	integer
note_evaluation_actuelle	integer
heure_supplementaires	varchar
augmentation_salaire_precedente	varchar
a_quitte_l_entreprise	varchar
nombre_participation_pee	integer
nb_formations_suivies	integer
distance_domicile_travail	float
niveau_education	integer
domaine_etude	varchar
frequence_deplacement	varchar
annees_depuis_la_derniere_promotion	integer
annes_sous_responsable_actuel	integer

prediction_logs	
id	integer
created_at	datetime
genre	varchar
statut_marital	varchar
departement	varchar
poste	varchar
domaine_etude	varchar
frequence_deplacement	varchar
augmentation_salaire_precedente	varchar
heure_supplementaires	varchar
revenu_mensuel	integer
satisfaction_employee_environnement	integer
satisfaction_employee_nature_travail	integer
satisfaction_employee_equipe	integer
satisfaction_employee_equilibre_pro_perso	integer
note_evaluation_precedente	integer
note_evaluation_actuelle	integer
niveau_hierarchique_poste	integer
niveau_education	integer
nb_formations_suivies	integer
nombre_participation_pee	integer
nombre_experiences_precedentes	integer
annees_dans_l_entreprise	integer
annees_dans_le_poste_actuel	integer
annees_depuis_la_derniere_promotion	integer
annes_sous_responsable_actuel	integer
age	float
annee_experience_totale	float
distance_domicile_travail	float
prediction	integer
probabilite_churn	float
interpretation	varchar

postgresql://localhost/churn_db

Tests unitaires et couverture

10 tests passés — **76%** create_db.py — **54%** couverture totale

```
pytest --cov=. --cov-report=html
```

10 tests au total

- **test_model.py** — pipeline de prédiction :
 - Chargement du modèle
 - Méthodes predict et predict_proba
 - Probabilité entre 0 et 1
 - Prédiction binaire (0 ou 1)
- **test_db.py** — base de données (SQLite en mémoire) :
 - Création des tables
 - Insertion et lecture d'un employé
 - Contrainte d'unicité
 - Insertion d'un log de prédiction
 - Remplissage automatique de created_at

```
===== tests coverage =====
__ coverage: platform darwin, python 3.12.2-final-0 __
```

Name	Stmts	Miss	Cover
conftest.py	0	0	100%
create_db.py	105	25	76%
features.py	104	104	0%
tests/test_db.py	50	0	100%
tests/test_model.py	20	0	100%
TOTAL	279	129	54%

```
===== 10 passed, 3 warnings in 2.06s =====
```


The background is a dark blue field filled with a complex network of glowing blue lines and dots. The lines connect various points, some of which are larger and brighter than others, creating a sense of depth and connectivity. The overall effect is reminiscent of a digital network or a molecular structure.

Merci